

Cartridge & Cloud - C# Coding Standards

Convenciones, arquitectura y calidad de código Unity

VRM Games / Blas Luis Rocha González

22/06/2026

Índice

1. Principio central	2
2. Namespaces	2
3. Nomenclatura	2
4. Estructura de clase	2
5. Reglas Unity	2
6. Diseño de dominio	2
7. Datos y persistencia	2
8. UI	2
9. Eventos	3
10. Logs	3
11. Fórmulas	3
12. Tests	3
13. Anti-patrones	3
14. Revisión de código	3

Proyecto: Cartridge & Cloud

Título técnico provisional: Cartridge & Cloud

Plataforma inicial: PC / Steam

Motor validado: Unity 6.3 LTS 6000.3.18f1 / C# / URP 17.3.0

Versión del documento: v0.3

Estado: Revisado en la baseline documental v0.4; contenido funcional vigente

Fuente conceptual: Enfoque_v0.6.md

Regla de interpretación: el proyecto se documenta como una preproducción nueva. Sprint 0 ha validado la fundación técnica (proyecto Unity, assemblies, escenas, tests y builds). Salvo cuando se cite evidencia de implementación, los sistemas de gameplay y contenido descritos en este documento continúan siendo especificación.

1. Principio central

La lógica de simulación no depende de GameObjects, paneles, textos ni botones. La presentación invoca casos de uso y representa resultados.

2. Namespaces

VRMGames.CartridgeAndCloud.Domain, .Application, .Infrastructure, .Presentation, .Content, .Editor, .Tests.

3. Nomenclatura

Elemento	Convención
Clases/métodos/propiedades	PascalCase
Interfaces	I + PascalCase
Campos privados	_camelCase
Variables/parámetros	camelCase
Constantes	PascalCase
Eventos	On + pasado/acción
IDs de contenido	snake_case con prefijo
ScriptableObjects	Nombre + Definition/Config
Estado runtime	Nombre + State/Data

4. Estructura de clase

Usings, namespace, summary, constantes, campos serializados, campos privados, propiedades, eventos, ciclo Unity, API pública y helpers privados.

5. Reglas Unity

- Evitar lógica pesada en **Update**.
- Cachear referencias.
- Desuscribir eventos.
- No usar búsquedas globales como dependencia normal.
- Validar referencias en **Awake**/editor.
- Coroutines solo para secuencias, no para ocultar estado.
- **ScriptableObject** no contiene progreso mutable.

6. Diseño de dominio

Entidades protegen invariantes. Operaciones que pueden fallar devuelven **Result<T>** o códigos tipados. Dinero utiliza entero en céntimos o decimal controlado; nunca float para contabilidad.

7. Datos y persistencia

IDs estables, DTOs explícitos, migraciones, normalización y serialización separada de las entidades cuando sea necesario. Todo campo persistente nuevo exige test de save/load.

8. UI

Controladores finos; localización por claves; dropdowns mapean a IDs y no a índices persistentes; async/loading con estados; no concatenar frases localizadas complejas.

9. Eventos

Eventos inmutables, nombres claros, sin cadenas de eventos circulares. Las operaciones críticas usan coordinación explícita, no solo event bus.

10. Logs

Categoría, contexto e ID. No escribir spam por frame. Errores de integridad usan error/exception; estados esperados usan warning/info.

11. Fórmulas

Parámetros en configuraciones, unidades documentadas, clamps explícitos y tests normales/extremos. No magic numbers.

12. Tests

AAA, nombres `Method_Scenario_Expected`, semillas fijas, dobles para reloj/RNG/repositorio y pruebas de invariantes monetarias/inventario.

13. Anti-patrones

God Manager, singleton global indiscriminado, lógica en UI, saves con referencias Unity, strings como estados, flags incompatibles, asignaciones por frame, catches vacíos y uso de `FindObjectOfType` como arquitectura.

14. Revisión de código

Compila sin warnings nuevos, cumple analyzers, tests, documentación de API pública relevante, no rompe saves, no introduce dependencia circular y actualiza trazabilidad.